

Rust

Perfectionnement

Haute Performance

Rust Haute Performance

Optimisation et Performance

Agenda du Jour 1

Horaire	Module	Durée
09:00 – 11:00	Module 1.1 – Profiling et benchmarking	2h
11:00 – 13:00	Module 1.2 – Optimisations mémoire	2h
14:00 – 16:00	Module 1.3 – Parallélisme data avec Rayon	2h
16:00 – 17:00	Module 1.4 – SIMD et vectorisation	1h

“ **Objectif** : Maîtriser l'optimisation Rust de bout en bout, du profiling à la vectorisation SIMD ”

Module 1.1 – Profiling et Benchmarking

Mesurer avant d'optimiser

Criterion : Benchmarks statistiquement fiables

```
use criterion::{criterion_group, criterion_main, Criterion, BenchmarkId};
fn bench_sorting(c: &mut Criterion) {
    let mut group = c.benchmark_group("sorting");
    for size in [100, 1_000, 10_000, 100_000] {
        let data: Vec<u64> = (0..size).rev().collect();
        group.bench_with_input(
            BenchmarkId::new("std_sort", size),
            &data,
            |b, data| {
                b.iter(|| {
                    let mut v = data.clone();
                    v.sort_unstable();
                    v
                })
            },
        );
        group.bench_with_input(
            BenchmarkId::new("rayon_par_sort", size),
            &data,
            |b, data| {
                b.iter(|| {
                    let mut v = data.clone();
                    v.par_sort_unstable();
                    v
                })
            },
        );
    }
    group.finish();
}
criterion_group!(benches, bench_sorting);
criterion_main!(benches);
```

Profiling CPU avec perf + flamegraph

```
# 1. Compiler avec symboles de debug
cargo build --release
# Ou dans Cargo.toml:
# [profile.release]
# debug = true

# 2. Profiling avec perf
perf record -g --call-graph dwarf ./target/release/my_app

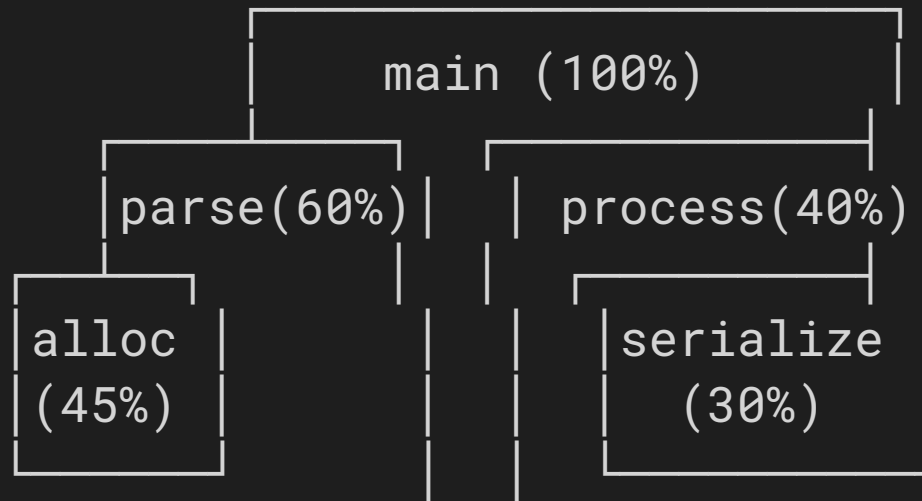
# 3. Générer le flamegraph
cargo install flamegraph
cargo flamegraph --release -- --my-args

# 4. Analyse
perf report --hierarchy
```

Astuce : Les plateaux larges du flamegraph indiquent les bottlenecks à optimiser en priorité

Lecture d'un flamegraph

Largeur = temps CPU consommé



“ Hauteur = profondeur de la stack ; largeur = temps passé ;
objectif = aplatir les plateaux larges ”

Profiling mémoire

heaptrack

```
heaptrack ./target/release/my_app  
heaptrack_gui heaptrack.my_app.*.gz
```

- Allocations totales
- Peak mémoire
- Leaks détectés
- Flamegraph des allocations

DHAT (valgrind)

```
// Cargo.toml  
// [features]  
// dhat-heap = ["dhat"]  
  
#[cfg(feature = "dhat-heap")]  
#[global_allocator]  
static ALLOC: dhat::Alloc = dhat::Alloc;  
  
fn main() {  
    #[cfg(feature = "dhat-heap")]  
    let _profiler = dhat::Profiler::new_heap();  
    // ...  
}
```


Atelier : Optimisation guidée par profiling

Objectif

Optimiser un programme de traitement de texte naïf à partir des données du profiler

Tâches

1. Benchmarker un programme de traitement de texte naïf
2. Profiler avec flamegraph → identifier les hotspots
3. Optimiser (réduire allocations, utiliser des slices)
4. Rebenchmarker → mesurer l'amélioration

Temps : 30 minutes

Module 1.2 – Optimisations Mémoire

Alignement et padding

```
// MAUVAIS : 24 bytes (padding)
struct Bad {
    a: u8,      // 1 byte + 7 padding
    b: u64,     // 8 bytes
    c: u8,      // 1 byte + 7 padding
}

// BON : 16 bytes (réordonné)
struct Good {
    b: u64,     // 8 bytes
    a: u8,      // 1 byte
    c: u8,      // 1 byte + 6 padding
}

// Vérifier avec
assert_eq!(std::mem::size_of::<Bad>(), 24);
assert_eq!(std::mem::size_of::<Good>(), 16);
```

Cache-friendly data structures (SoA vs AoS)

AoS (Array of Structs)

```
struct Particle {  
    x: f64, y: f64, z: f64,  
    mass: f64,  
    name: String, // 24 bytes!  
}  
let particles: Vec<Particle> = ...;  
  
// Itérer sur x,y,z charge aussi  
// mass et name en cache → gaspillage
```

SoA (Struct of Arrays)

```
struct Particles {  
    x: Vec<f64>,  
    y: Vec<f64>,  
    z: Vec<f64>,  
    mass: Vec<f64>,  
    name: Vec<String>,  
}  
  
// Itérer sur x,y,z : données  
// contiguës en mémoire → cache hits
```

“ SoA = **3-10x plus rapide** pour les boucles sur un sous-ensemble de champs ”

Arena Allocators avec bumpalo

```
use bumpalo::Bump;

fn process_batch(data: &[RawEvent]) -> Vec<ProcessedEvent> {
    // Arena : allocation ultra-rapide, libération en bloc
    let arena = Bump::new();

    let mut results = Vec::new();
    for event in data {
        // Allocation dans l'arena (pas de free individuel)
        let parsed = arena.alloc(parse_event(event));
        let enriched = arena.alloc(enrich(parsed));

        if enriched.is_relevant() {
            results.push(enriched.to_owned());
        }
    }
    // Arena libérée d'un coup ici (drop)
    results
}
```

SmallVec et Small String Optimization

```
use smallvec::SmallVec;

// Vec<u8> : toujours sur le heap (24 bytes sur le stack)
let v: Vec<u8> = vec![1, 2, 3];

// SmallVec : inline jusqu'à N éléments, heap au-delà
let sv: SmallVec<[u8; 16]> = smallvec![1, 2, 3]; // Pas d'allocation heap!

// Compact strings (smol_str, compact_str)
use compact_str::CompactString;
let s: CompactString = CompactString::new("short"); // Inline (pas de heap)
let s: CompactString = CompactString::new("this is a longer string"); // Heap
```

Quand utiliser : Collections souvent petites (tags, labels, petits buffers), structures allouées massivement

Module 1.3 – Parallélisme Data avec Rayon

Rayon : Parallel iterators

```
use rayon::prelude::*;

// Séquentiel
let sum: u64 = data.iter().map(|x| expensive_compute(x)).sum();

// Parallèle (une ligne de changement!)
let sum: u64 = data.par_iter().map(|x| expensive_compute(x)).sum();

// Tri parallèle
let mut data = vec![5, 3, 1, 4, 2];
data.par_sort_unstable();

// Filter + map parallèle
let results: Vec<_> = events.par_iter()
    .filter(|e| e.level == "ERROR")
    .map(|e| process_error(e))
    .collect();

// Chunking explicite
data.par_chunks(1000)
    .for_each(|chunk| {
        process_chunk(chunk);
    });
```


Rayon : join et scope

```
use rayon::join;

// Fork-join : deux tâches en parallèle
let (result_a, result_b) = rayon::join(
    || compute_hash(&data_a),
    || compute_hash(&data_b),
);

// Scope : parallélisme structuré avec emprunt
fn parallel_sum(data: &[u64]) -> u64 {
    if data.len() <= 1000 {
        return data.iter().sum(); // Seuil séquentiel
    }

    let mid = data.len() / 2;
    let (left, right) = data.split_at(mid);

    let (sum_l, sum_r) = rayon::join(
        || parallel_sum(left),
        || parallel_sum(right),
    );
    sum_l + sum_r
}
```

Atelier : Pipeline de traitement parallèle

Objectif

Traiter un dataset de 1M d'événements réseau en parallèle

Tâches

1. Charger depuis un fichier CSV (rayon parallel IO)
2. Parser chaque ligne (`par_iter` + `map`)
3. Filtrer les anomalies (`par_iter` + `filter`)
4. Agréger par source IP (`par_iter` + `fold` + `reduce`)
5. Benchmarker séquentiel vs parallèle avec Criterion

Temps : 30 minutes

Module 1.4 – SIMD et Vectorisation

Auto-vectorisation par le compilateur

```
// Le compilateur peut vectoriser automatiquement :  
fn dot_product(a: &[f64], b: &[f64]) -> f64 {  
    a.iter().zip(b.iter()).map(|(x, y)| x * y).sum()  
}  
  
// Vérifier avec :  
// RUSTFLAGS="-C target-cpu=native" cargo build --release  
// cargo asm my_crate::dot_product
```

Conditions pour l'auto-vectorisation

- Boucles simples sans dépendances de données
- Types alignés (f32, f64, u32, etc.)
- Pas de branches imprévisibles dans la boucle
- Compiler avec -C target-cpu=native

SIMD explicite (aperçu)

```
#[cfg(target_arch = "x86_64")]
use std::arch::x86_64::*;

// Somme de 4 floats en une instruction
unsafe fn sum_f32x4(a: &[f32; 4], b: &[f32; 4]) -> [f32; 4] {
    let va = _mm_loadu_ps(a.as_ptr());
    let vb = _mm_loadu_ps(b.as_ptr());
    let vc = _mm_add_ps(va, vb);
    let mut result = [0f32; 4];
    _mm_storeu_ps(result.as_mut_ptr(), vc);
    result
}
```

Cas d'usage	Gain typique
Parsing (bytes scanning)	2-4x
Hashing (xxhash, wyhash)	3-5x
Compression (lz4, zstd)	2-3x
Calcul numérique	4-8x

Récapitulatif Jour 1

Points clés

- Profiling : Criterion, flamegraph, heaptrack
- Mémoire : alignement, SoA, arenas, SmallVec
- Parallélisme : Rayon (`par_iter`, `join`, `scope`)
- SIMD : auto-vectorisation + intrinsics

Demain : Programmation Concurrente Avancée

